MODUL 6 · TYPESCRIPT DALAM NEXT.JS

PageProps & searchParams



Memberi tipe pada props yang diberikan Next.js 16 kepada setiap halaman

Page Menerima Props dari Next.js

Berbeda dengan komponen biasa di sub-bab 4: yang memanggil page adalah Next.js, bukan kita.

Komponen biasa: kita yang memanggil

<Tombol judul="Simpan" /> — kita sendiri yang menentukan props yang dikirim

Page: Next.js yang memanggil

File page.tsx tidak dipakai sebagai tag; Next.js mengisi propsnya dari URL

Dua props yang disediakan: params dan searchParams

Tulis hanya yang kamu butuhkan

Dari Mana Asalnya params dan searchParams ?

params berasal dari bagian path yang dinamis. searchParams berasal dari query string setelah tanda ?.

URL	File	params	searchParams
/blog/belajar-ts	app/blog/[slug]/page.tsx	{ slug: "belajar-ts" }	{ }
/shop?page=2&sort=asc	app/shop/page.tsx	{ }	{ page: "2", sort: "asc" }
/shop?tag=a&tag=b	app/shop/page.tsx	{ }	{ tag: ["a", "b"] }

params dan searchParams adalah *Promise*

Di Next.js 16, keduanya tidak bisa dibaca langsung. Nilainya harus ditunggu dulu.

Server Component: pakai await

Fungsi page ditulis async, lalu const { slug } = await params

Client Component: pakai use()

Client Component tidak bisa async, jadi memakai fungsi use dari React

Lupa menunggu = error pada tipe

TypeScript langsung menandai, seperti di kode berikutnya

Membaca Tipe params

params: Promise<{ slug: string }> dibaca dari luar ke dalam.

Promise<...>: dibungkus Promise
Nilainya belum langsung tersedia; harus ditunggu dengan await

{ slug: string }: bentuk isinya
Objek dengan satu properti slug bertipe string

slug harus sama dengan nama folder [slug]
Tipe manual ini tidak dicek terhadap nama folder

Contoh Tipe params Manual

Memakai pola props dari sub-bab 4: destructuring di kiri, tipe inline di kanan.

app/blog/[slug]/page.tsx

```
export default async function Page({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params // benar: ditunggu dulu
  const x = params.slug         // salah: params masih Promise
  // Error: Property 'slug' does not exist on type
  // 'Promise<{ slug: string; }>'.

  return <h1>Blog Post: {slug}</h1>
}
```

searchParams: Kenapa string | string[] | undefined?

Tipenya: Promise<{ [key: string]: string | string[] | undefined }>

string
Nilai dari URL selalu teks: ?page=2 menghasilkan "2", bukan angka 2

string[]
Kunci yang muncul berulang: ?tag=a&tag=b menghasilkan ["a", "b"]

undefined
Kunci yang tidak ada di URL

{ [key: string]: ... }
Objek dengan kunci apa saja yang bertipe string, karena kita tidak tahu query apa yang dikirim pengguna

Contoh Tipe searchParams Manual

Nilai bawaan pada destructuring menghilangkan undefined, tetapi bukan string[].

app/shop/page.tsx

```tsx
export default async function Page({
  searchParams,
}: {
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}) {
  const { page = "1", sort = "asc" } = await searchParams
  // page dan sort bertipe string | string[]
  // (nilai bawaan "1" dan "asc" menghilangkan undefined)

  return <p>Halaman {page}, urutan {sort}</p>
}
```

Contoh Menangani string | string[]

TypeScript memaksa kita memastikan hanya ada satu nilai sebelum memakainya sebagai string.

app/shop/page.tsx

```
const { page = "1" } = await searchParams

const p: string = page
// Error: Type 'string | string[]' is not assignable to type 'string'.
// Type 'string[]' is not assignable to type 'string'.

// Perbaikan: pastikan hanya satu nilai
const halaman = Array.isArray(page) ? page[0] : page
// halaman bertipe string, aman dipakai
```

PageProps: Tipe yang Digenerate Next.js

Tidak perlu menulis tipe params dan searchParams sendiri.

Helper global: tanpa import

Digenerate Next.js saat next dev, next build, atau next typegen

Ditulis PageProps<"/blog/[slug]">

Isi di dalam <> adalah path route, sama dengan struktur foldernya

params dan searchParams langsung bertipe

params mengikuti nama folder dinamis. searchParams sudah bertipe Promise berisi string | string[] | undefined

Contoh Memakai PageProps

Satu parameter props, tanpa menulis tipe params dan searchParams.

app/blog/[slug]/page.tsx

```
export default async function Page(props: PageProps<"/blog/[slug]">) {
  const { slug } = await props.params
  const query = await props.searchParams

  const { id } = await props.params
  // Error: Property 'id' does not exist on type '{ slug: string; }'.

  return <h1>Blog Post: {slug}</h1>
}
```

Tipe Manual vs PageProps

Perbedaan yang paling penting: siapa yang memastikan nama params benar.

	Tipe manual	PageProps
Siapa yang menulis tipe	Kita sendiri	Digenerate dari struktur folder
Nama params cocok dengan folder?	Tidak dicek. Bisa salah tanpa error	Dicek. Salah nama langsung error
Setup	Tidak ada	Otomatis lewat next dev, next build, atau next typegen
Di course ini	Dipahami, karena banyak dokumentasi dan tutorial masih memakainya	Dipakai sebagai standar

Contoh Tipe Manual yang Salah Nama

Folder bernama [id], tetapi tipenya menyebut slug. Hanya PageProps yang menangkapnya.

app/produk/[id]/page.tsx

```
// Manual: nama tidak cocok dengan folder [id]
export default async function Page({ params }: { params: Promise<{ slug: string }> }) {
  const { slug } = await params
  // tidak ada peringatan, bahkan saat next build
  // nilai slug tidak akan terisi saat dijalankan
}

// Dengan PageProps
export default async function Page(props: PageProps<"/produk/[id]">) {
  const { slug } = await props.params
  // Error: Property 'slug' does not exist on type '{ id: string; }'.
}
```

Contoh Client Component dengan use()

Client Component tidak bisa async, jadi Promise dibaca dengan use dari React.

app/cari/page.tsx

```
'use client'

import { use } from 'react'

export default function Page({
  searchParams,
}: {
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>
}) {
  const { query } = use(searchParams) // bukan await
  return <p>Pencarian: {query}</p>
}
```

Batasan yang Perlu Diingat

Tipe bukan validasi

?page=abc tetap bertipe string, jadi TypeScript tidak protes. Number("abc") menghasilkan NaN. Nilai dari URL bisa diisi siapa saja, jadi tetap perlu divalidasi saat aplikasi berjalan (kita bahas dengan Zod).

searchParams membuat halaman dinamis

Menurut dokumentasi Next.js, searchParams hanya diketahui saat ada request, sehingga halaman yang memakainya dirender saat request, bukan di awal.